

Week 6 - Class 11: Functions: Basics, Parameters, Return Values

Why This Matters

Without functions, all logic lives in `main()`. Functions let you:

- Break problems into reusable pieces.
- Avoid copy-paste code.
- Make code clearer and more maintainable.

In C++14, functions are declared with return type, name, parameters, and body.

Anatomy of a Function

General form:

```
return_type functionName(parameters){ body return value; }
```

```
int add(int a, int b) {  
    return a + b;  
}
```

Calling a Function

Call functions by name and arguments.

```
int main() {  
    int sum = add(3,4);  
    cout << "Sum = " << sum << "\n";  
}
```

Parameters (Inputs)

Functions can take inputs. Pass by value makes a copy.

```
void greet(string name) {  
    cout << "Hello, " << name << "!\n";  
}
```

Return Values (Outputs)

Functions can return results.

```
int square(int x) {  
    return x * x;  
}
```

```
void printLine() {  
    cout << "-----\n";  
}
```

Function Prototypes (Forward Declarations)

A function prototype is a forward declaration that tells the compiler about the function's name, return type, and parameter types, but not its body.

Why needed? C++ reads top-to-bottom. If you call a function before its definition, the compiler doesn't know it exists. A prototype is a promise: "trust me, this function exists and will be defined later."

Prototype Example (Without and With)

```
// Error: square not declared yet  
int main() {  
    cout << square(5);  
}  
  
int square(int x) {  
    return x * x;  
}
```

```
}  
  
// Works with prototype  
int square(int x);    // prototype  
  
int main() {  
    cout << square(5);  
}  
  
int square(int x) {  
    return x * x;  
}
```

When to Use Prototypes

- If the function is defined before it's called: no prototype needed.
- If the function is defined after it's called: you need a prototype.

Later, with header files, prototypes go in headers and definitions in .cpp files. For now, think of prototypes as a way to keep main() at the top while defining functions later.

Analogy

Think of a prototype like an author introducing a character before they appear. It tells the compiler: this function will show up later.

ASCII Flow: Function Call

main() -> call add(3,4) -> function body runs -> return result -> back to main

Common Mistakes

1. Forgetting return in non-void.
2. Return type mismatch.
3. Passing wrong types.
4. Forgetting prototypes when defining after main.

Exercises

1. Write square(int).
2. Write max(int,int).
3. Write printGreeting() void.
4. Use add, square, max in main.
5. Challenge: sum 1..n.

Project Tie-In

Build Function Library:

- math.cpp with add, square, max.
- Prototypes in main.cpp.
- Organize code into reusable chunks.

This prepares you for header files and modular projects.